

Санкт-Петербургский государственный университет

Кафедра информационно-аналитических систем

Группа 23.Б08-мм

Исследование эффективности метода Spectrum-to-Signal в малых языковых МОДЕЛЯХ

Речкин Вадим Сергеевич

Отчёт по учебной практике
в форме «Эксперимент»

Научный руководитель:
ст. преподаватель кафедры ИАС, к.ф.-м.н., Азимов Р. Ш.

Санкт-Петербург
2026

Оглавление

Введение	3
1. Постановка задачи	4
2. Обзор	5
2.1. Обзор предметной области	5
2.2. Обзор предложенной статьи	6
2.3. Модели, используемые в работе	11
3. Описание решения	12
3.1. Описание экспериментальной задачи	12
3.2. Создание наборов данных	12
3.3. Создание верификатора	16
3.4. Обучение модели GPT2	17
4. Эксперимент	20
4.1. Условия эксперимента	20
4.2. Исследовательские вопросы	20
4.3. Метрики	20
4.4. Результаты	21
4.5. Анализ результатов	22
Заключение	24
Список литературы	25

Введение

Компания OpenAI¹ разработала подход к обучению больших языковых моделей с использованием длинных цепочек рассуждений, что привело к заметному росту качества при решении логических задач. С того момента вышло много различных языковых моделей с рассуждением, и главным методом повышения качества было масштабирование моделей.

Сложился стереотип, что модели с небольшим числом параметров лишены хорошей способности к рассуждению. Команда учёных из WeiboAI² опровергла этот стереотип, представив модель VibeThinker-1.5B[8], которая по современным меркам имеет небольшое число параметров — 1.5 миллиарда. Эта модель превосходит многие известные модели, такие как DeepSeek R1[1] и некоторые модели семейств Qwen3 и Llama, в математических тестах и ряде других задач, требующих логических рассуждений. Такой результат получился благодаря предложенному методу обучения Spectrum-to-Signal[8] (SSP).

На основе анализа этой статьи выдвигается следующая гипотеза: можно ли добиться значительного улучшения результатов, применив предложенный метод обучения к ещё меньшим моделям с числом параметров порядка сотен миллионов.

В данной работе будет проведён анализ метода обучения Spectrum-to-Signal[8] (SSP), а также эксперимент по обучению модели GPT2[4] для генерации решений задач на языке Python с использованием данного метода и сравнение этого подхода с обычным дообучением на обработанных наборах данных APPS³ и Magicoder-OSS-Instruct-75K⁴, а также дообучением с фазой RL в виде GRPO[2].

¹Сайт компании OpenAI: <https://openai.com> (дата обращения: 10.03.2026)

²GitHub компании WeiboAI <https://github.com/WeiboAI> (дата обращения: 10.03.2026)

³Ссылка на набор данных APPS: <https://huggingface.co/datasets/codeparrot/apps> (дата обращения: 10.03.2026)

⁴Ссылка на набор данных Magicoder-OSS-Instruct-75K <https://huggingface.co/datasets/ise-uiuc/Magicoder-OSS-Instruct-75K> (дата обращения: 10.03.2026)

1. Постановка задачи

Целью работы является исследование эффективности метода Spectrum-to-Signal (SSP) для малых языковых моделей. Для её выполнения были поставлены следующие задачи:

1. Провести обзор предложенной статьи и метода обучения SSP.
2. Подготовить наборы данных для задачи генерации решений задач на языке Python.
3. Дообучить модель GPT2 с использованием только тонкой настройки.
4. Дообучить модель GPT2 с использованием тонкой настройки и фазы RL в виде GRPO.
5. Дообучить модель GPT2 с использованием предложенного метода SSP.
6. Сравнить результаты обучения, сделать выводы об эффективности метода SSP

2. Обзор

В этом разделе будут проведёны обзоры предметной области, предложенной статьи, в частности метода обучения SSP и моделей, которые будут использоваться в эксперименте.

2.1. Обзор предметной области

В данной работе для сравнения с предложенным методом SSP будут использоваться более ранние подходы. Поэтому до проведения обзора данной статьи, рассмотрим эти алгоритмы ниже.

2.1.1. Оптимизация относительной политики группы

Оптимизация относительной политики группы (Group-Relative Policy Optimization[2], GRPO) — алгоритм обучения с подкреплением, представленный учёными из команды DeepSeek. Этот алгоритм является усовершенствующей версией PPO[6], а именно GRPO устраняет необходимость второй модели для оценки первой. Вместо предсказания абсолютной ценности состояний используется относительное преимущество, вычисляемое по группе ответов на один и тот же вопрос.

Рассмотрим алгоритм GRPO

1. **Вычисление относительного преимущества:** Для вопроса q генерируется G ответов, каждому из которых внешний верификатор назначает награду. В результате получаем набор вознаграждений $\{r_i\}_{i=1}^G$. Далее вычисляется относительное преимущество:

$$A_i = \frac{r_i - \mu}{\sigma} \text{ где}$$

- μ — среднее арифметическое вознаграждение ответов $\{r_i\}_{i=1}^G$.
- σ — стандартное отклонение ответов $\{r_i\}_{i=1}^G$.

2. **Целевая функция:** Далее преимущество будет использоваться в целевой функции, рассмотрим эту функцию:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{(q,y) \sim D} \left[\mathbb{E}_{\{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(y|q)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{j=1}^{|y_i|} (\min(r_{i,j}(\theta) \cdot A_i(q), \text{clip}(r_{i,j}(\theta), 1 - \epsilon, 1 + \epsilon) \cdot A_i(q))) \right] \right]$$

Где:

- θ — параметры модели.
- θ_{old} — параметры модели до обновления.
- D — набор задач.
- G — количество ответов.
- $A_i(q)$ — преимущество i -го ответа на вопрос q .
- $r_{i,j}(\theta)$ — отношение вероятностей токена до и после обновления весов модели.
- ϵ — параметр усечения.
- $\text{clip}(x, a, b)$ — функция усечения, возвращающая x , если x находится в интервале $[a, b]$, иначе a или b в зависимости от того, что ближе.

GRPO эффективен в задачах, требующих развёрнутых рассуждений: отказ от модели-критика позволяет увеличить размер группы ответов и длину рассуждений.

2.2. Обзор предложенной статьи

Она основана на архитектуре Qwen2.5-Math-1.5B[7], её дообучение, по словам авторов, обошлось примерно в 7800 долларов, что сравнительно мало для полученных результатов.

На рисунке 1 представлена схема обучения модели Qwen2.5-Math. При рассмотрении метода SSP можно заметить его сходство с этим подходом. Однако такая схема обучения является общим для разных версий Qwen2.5-Math, тогда как авторы статьи показали результаты, превосходящие соразмерную версию Qwen2.5-Math.

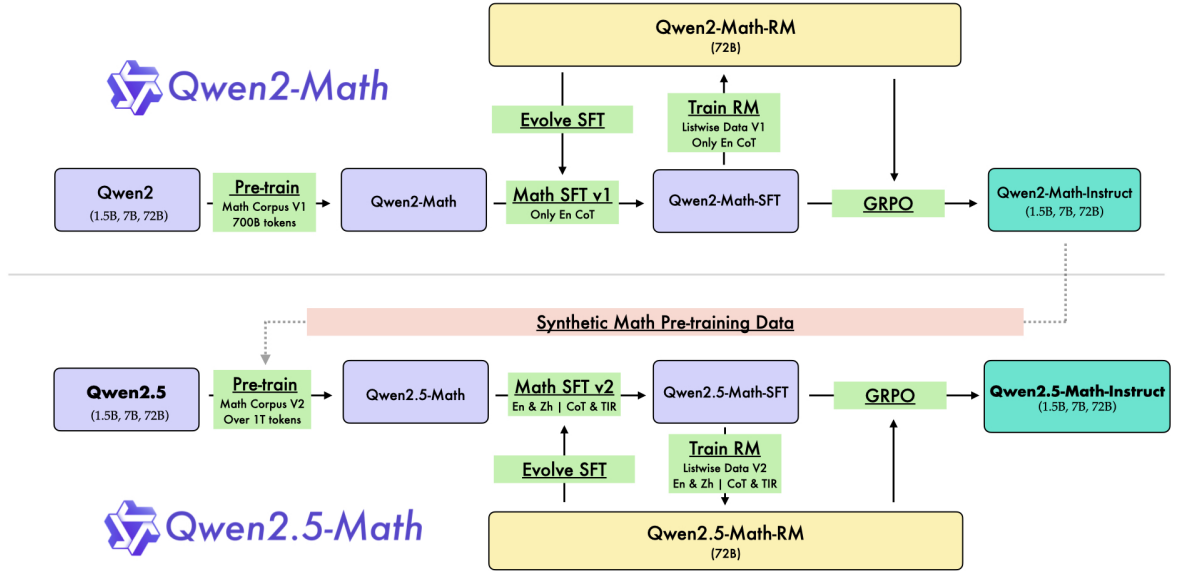


Рис. 1: Метод обучения Qwen2.5-Math

2.2.1. Увеличение разнообразия модели

Чтобы полностью понять принцип работы метода обучения SSP, проведём разбор о важности разнообразия модели.

Для начала рассмотрим метрику Pass@K , которая показывает вероятность, что из K сгенерированных ответов, хотя бы один будет правильным:

$$\text{Pass@K} = \mathbb{E}_{x \sim D, y_{i=1}^K \sim \pi_{\theta}(y|x)} [\max\{R(x, y_1), \dots, R(x, y_K)\}], \text{ где:}$$

- D — набор задач.
- $\pi_{\theta}(y|x)$ — условное распределение ответов модели.
- $R(x, y)$ — бинарная функция вознаграждения.

Стандартный подход к дообучению моделей рассуждения заключается в том, что сначала выполняется тонкая настройка модели (SFT), а затем применяется обучение с подкреплением (RL) для дальнейшего улучшения качества. После процесса SFT обычно сохраняют веса модели, при которых достигается наилучшее значение метрики Precision или Pass@1 на валидационном наборе.

Такой подход является неоптимальным, поскольку он ограничивает потенциальный прогресс на этапе обучения с подкреплением. Авторы статьи утверждают, что максимизация метрики Pass@K позволяет улучшить разнообразие модели, тем самым подготавливая для фазы RL более гибкую модель.

2.2.2. Метод обучения ”Спектр — сигнал”

Процесс дообучения модели VibeThinker-1.5B состоит из двух фаз: сначала модель дообучается с помощью тонкой настройки (SFT) с целью максимизации метрики Pass@K, а затем на фазе RL усиливаются правильные ответы.

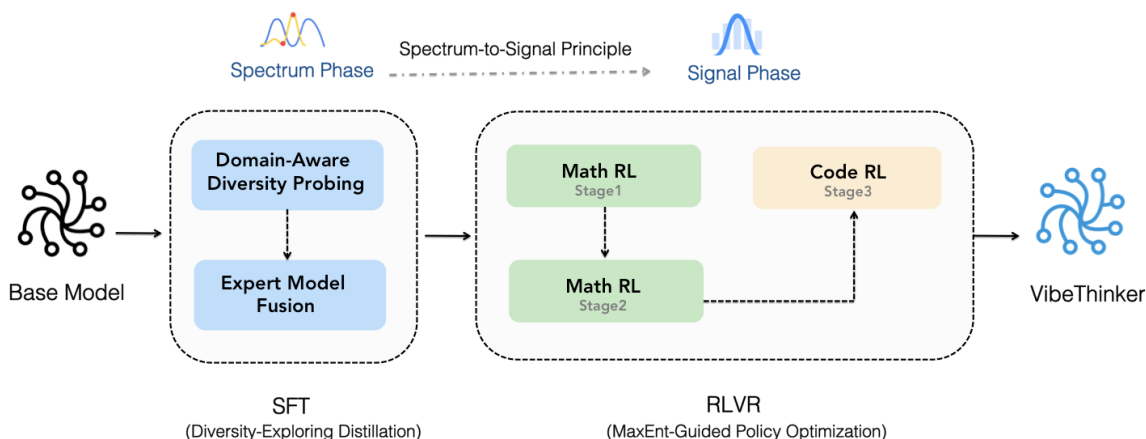


Figure 3. The Training Pipeline of VibeThinker-1.5B

Рис. 2: Метод SSP для дообучения модели

Чтобы понять, в чём заключаются нововведения метода, рассмотрим каждую из этих фаз.

2.2.3. Дистилляция с исследованием разнообразия

Фазу тонкой настройки SFT авторы называют дистилляцией с исследованием разнообразия. Этот процесс состоит из двух этапов:

- **Исследование разнообразия с учётом предметной области:** на этом этапе модель дообучается на большом наборе данных, а

метрика Pass@K периодически вычисляется на специальных оценочных наборах данных, соответствующих отдельным доменам. Каждый домен является набором примеров из определённой области. В текущей статье были использованы следующие домены: алгебра, геометрия, математический анализ и математическая статистика. Каждый раз, когда значение Pass@K в некотором домене превышает предыдущее лучшее значение для этого домена, сохраняются веса модели.

- **Слияние экспертных моделей:** После дообучения получаем набор моделей $M_1, M_2, \dots, M_n$, каждая из которых показывает наилучшее значение Pass@K в своём домене. Далее параметры этих моделей объединяются, и в результате получается одна экспертная модель:

$$M = \sum_{i=1}^n w_i \cdot M_i, \text{ где } \sum_{i=1}^n w_i = 1 \text{ и } w_i \geq 0.$$

В обзоре статьи $w_i = \frac{1}{n}$.

По словам авторов, такой подход не просто создаёт модель с широким разнообразием, но и увеличивает потенциальный прогресс на этапе обучения с подкреплением.

2.2.4. Оптимизация политики на основе максимальной энтропии

Алгоритм оптимизации политики на основе максимальной энтропии (MaxEnt-Guided Policy Optimization, MGPO) — это алгоритм обучения с подкреплением, представляющий собой модификацию GRPO. Авторы статьи утверждают, что ценность от обучаемой задачи максимальная, когда текущая политика имеет максимальную энтропию в её решении.

Далее рассмотрим подробности работы алгоритма MGPO.

1. **Принцип максимальной энтропии:** Для вопроса q генерируется G ответов, что порождает бинарное распределение решений

(верное или нет). Тогда эмпирическая вероятность правильного ответа на вопрос q равна:

$$p_c(q) = \frac{1}{G} \sum_{i=1}^G I(r_i = 1), \text{ где}$$

- r_i — вознаграждение, полученное за i -й ответ на вопрос q .
- I — индикаторная функция.

Максимальная энтропия достигается при $p_c(q) = 0.5$. В этом случае модель не склоняется устойчиво ни к правильным, ни к неправильным ответам, поэтому такая задача считается наиболее полезной для дальнейшей оптимизации.

2. **Регуляризация отклонения энтропии:** Чтобы измерить расстояние от положения с максимальной энтропией, авторы используют KL-дивергенцию с текущим распределением модели и идеальным, с вероятностью $p_0 = 0.5$:

$$D_{ME}(p_c||p_0) = p_c(q) \log \frac{p_c(q)}{p_0} + (1 - p_c(q)) \log \frac{1 - p_c(q)}{1 - p_0}.$$

После вычисления этого расстояния задаче назначается следующий вес:

$$w_{ME}(p_c(q)) = \exp(-\lambda \cdot D_{ME}(p_c||p_0)), \text{ где } \lambda \text{ — гиперпараметр.}$$

Если взять $\lambda = 0$, то метод совпадёт с GRPO. Чем больше λ , тем меньший вес получают как слишком простые, так и слишком сложные задачи.

3. **Функция потерь:** Вес, полученный в предыдущем пункте, применяем к преимуществу $A_i(q)$:

$$A'_i(q) = w_{ME}(p_c(q)) \cdot A_i(q).$$

Итоговая функция потерь:

$$\mathcal{J}_{\text{MGPO}}(\theta) = \mathbb{E}_{(q,y) \sim D} \left[\mathbb{E}_{\{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(y|q)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{j=1}^{|y_i|} (\min(r_{i,j}(\theta) \cdot A'_i(q), \text{clip}(r_{i,j}(\theta), 1 - \epsilon, 1 + \epsilon) \cdot A'_i(q))) \right] \right]$$

Где:

- θ — параметры модели.
- θ_{old} — параметры модели до обновления.
- D — набор задач.
- G — количество ответов.
- $A'_i(q)$ — преимущество i -го ответа на вопрос q .
- $r_{i,j}(\theta)$ — отношение вероятностей токена до и после обновления весов модели.
- ϵ — параметр усечения.

2.3. Модели, используемые в работе

В этом разделе будут описаны модели, которые были использованы в работе: для обучения и для создания набора данных

2.3.1. Обзор модели GPT2

GPT2 — авторегрессионная модель, основанная на архитектуре decoder-only transformer. Размер её контекстного окна составляет 1024 токена. Для кодирования позиции токенов использовались обучаемые абсолютные эмбединги, поэтому при обучении модель может обрабатывать и сгенерировать 1024 токена в сумме.

2.3.2. Обзор модели ...

3. Описание решения

В данном разделе будет описана экспериментальная задача, представлены схемы обучения модели, а также рассмотрены вспомогательные шаги по созданию и обработке наборов данных.

3.1. Описание экспериментальной задачи

В данной работе будет проведён эксперимент по обучению лёгкой модели GPT2 для решения задач на языке Python. На вход модели будет подаваться условие задачи на английском языке и несколько примеров, по которым модель должна сгенерировать своё рассуждение и после этого написать решение задачи на языке Python.

Сам эксперимент заключается в обучении модели тремя способами:

1. Тонкая настройка без фазы RL
2. Тонкая настройка с фазой RL в виде GRPO
3. Предложенный метод: SFT + MGPO

3.2. Создание наборов данных

В качестве исходных наборов данных решено было взять APPS⁵ и Magicoder-OSS-Instruct-75K⁶.

APPS содержит 5000 заданий, из которых примерно 75% имеют различные решения, остальные 25% — не размечены. Если преобразовывать задачи в промпт, то для большинства из них суммарная длина условия и даже самого короткого решения превышает 1024 токена. Также в наборе данных отсутствует разметка с рассуждением, поэтому исходный набор данных будет использован как основа для создания наборов, используемых в работе, а именно для фазы SFT и для создания оценочных доменов.

⁵Ссылка на набор данных APPS: <https://huggingface.co/datasets/codeparrot/apps> (дата обращения: 10.03.2026)

⁶Ссылка на набор данных Magicoder-OSS-Instruct-75K <https://huggingface.co/datasets/ise-uiuc/Magicoder-OSS-Instruct-75K> (дата обращения: 10.03.2026)

Magocoder-OSS-Instruct-75K имеет 75 тысяч заданий с решениями на разных языках программирования, но почти половина из них на языке Python. Данный набор данных будет использоваться для фазы RL, но предварительно требует обработки.

3.2.1. Набор данных для обучения модели

На данном этапе нужно создать набор данных, на котором будет проходить основное дообучение модели. Для того, чтобы преобразовать набор данных, было решено обработать все задачи с помощью большой языковой модели. Поскольку многие известные большие языковые модели имеют ограниченный доступ к API, было решено использовать Mistral-7B-Instruct-v0.3[5], развернув её на платформе Kaggle. Выбор пал именно на эту модель, потому что она обеспечивает баланс между скоростью, точностью и количеством параметров.

Промпт для обработки задачи был следующим:

```
"You are an experienced senior python teacher. "  
"Our mission is to train a model with a small context window to reason. "  
"Your task is to study the condition of the problem and its solution, "  
"and then generate a short, clear condition with two examples, "  
"a little point-by-point reasoning that helps solve the problem.\n"  
f"question:\n {question}\n"  
f"solution:\n {solution}\n"  
"Requirements for tasks:\n"  
"1. The task conditions should be clear and short in the following format:\n"  
"Question:\n...\nExample 1:\nInput: ...; Output: ...\n"  
"2. Arguments should be point-by-point, short, helping to solve the problem.\n"  
"3. The solution should be clear. The names of variables and functions are clear. "  
"There should be no comment.\n"  
"4. The limitation of the contextual window of the model is 1024 tokens.\n"  
"5. Your answer should be complete, not cut off, but finished.\n"  
"Response format (strictly follow)\n"  
"{\n"  
"  \"question\": \"Task condition\",\\n"  
"  \"example\": \"Input: ...; Output: ...\",\\n"  
"  \"reasoning\": \"Point-by-point reasoning\",\\n"  
"  \"solution\": \"Code on Python\"\\n"  
"}\n"
```

Применяем этот промпт к каждой задаче из набора данных и сохраняем результаты в файл.

3.2.2. Оценочные наборы данных

Далее нужно разбить наши задачи на домены, каждый из которых будет использован для оценки модели в своей области. В данной работе решено было разделить на три домена:

1. Алгоритмы
2. Структуры данных
3. Математика

Читатели могут обратить внимание, что алгоритмы и структуры данных являются связанными темами, но в первом домене акцент сделан на написании именно популярных алгоритмов и их производных, к примеру сортировок, различных подсчётов, использования рекурсии и т.п., а во втором домене акцент сделан на структурах данных, а именно на использовании и возможной реализации их в алгоритме.

Было решено создать по 60 задач в каждом домене, для генерации трёх задач, по одной на каждый домен.

```
"You are an experienced senior python teacher. "  
"Your task is to create an estimated dataset of learning tasks and their solutions in  
python.\n\n"  
"Requirements for tasks:\n"  
"1. Write one problem each on the following topics: "  
f"algorithms (use subtopic - {words[0]}), "  
f"data structures (use subtopic - {words[1]}), "  
f"mathematics (use subtopic - {words[2]}).\n"  
"2. Task difficulty: medium.\n"  
"3. Task format: Clear condition, example data, reasoning, clear decision.\n"  
"4. The task must be self-sufficient and unique. "  
"Preferably a new idea or an unusual, but clear and understandable condition.\n\n"  
"Requirements for the solution:\n"  
"1. Working code in Python 3.8+.\n"  
"2. Use functions and data structures from the standard library. "  
"Math, numpy, and scipy libraries can be used in math problems.\n"  
"3. Readability and PEP8 compliance.\n"  
"4. The writing style should match the style of the attached examples.\n"  
"Your answer must be a dictionary.\n"  
"Response format (strictly follow)\n"  
"```json\n"  
"{\n\n"
```

```

" \"algorithm\": {\\n"
" \"question\": \"Task condition\",\\n"
" \"example\": \"Input: ...; Output: ...\",\\n"
" \"reasoning\": \"Point-by-point reasoning\",\\n"
" \"solution\": \"Code on Python\"\\n"
" },\\n"
" \"data structure\": {\\n"
" \"question\": \"Task condition\",\\n"
" \"example\": \"Input: ...; Output: ...\",\\n"
" \"reasoning\": \"Point-by-point reasoning\",\\n"
" \"solution\": \"Code on Python\"\\n"
" },\\n"
" \"mathematic\": {\\n"
" \"question\": \"Task condition\",\\n"
" \"example\": \"Input: ...; Output: ...\",\\n"
" \"reasoning\": \"Point-by-point reasoning\",\\n"
" \"solution\": \"Code on Python\"\\n"
" }\\n"
"}\\n"
"Example of a task:\\n"
f"{str(dict(example_task))}\\n"

```

В массиве `words` находятся слова, которые являются подтемами для каждого домена. Они нужны для того, чтобы разнообразить ответы модели и уменьшить число повторений. По такой же причине выбирается случайная задача из набора данных в качестве примера.

После генерации наборы данных сохраняются в файлы.

3.2.3. Набор данных для фазы RL

Как уже было сказано выше, для фазы RL использовался набор данных `Magocoder-OSS-Instruct-75K`. В начальном виде набор данных не подходит для этой задачи: нужно разделить исходное условие на чёткое условие и примеры, а также создать ещё несколько тестовых примеров, которые модель не увидит.

Для обработки набора данных модель использовалась та же, `Mistral-7B-Instruct-v0.3`, промпт для обработки выглядел следующим образом:

```

"You are an experienced senior python teacher. "
"Our task is to rework the data set for the RL phase in a small model with a small
context window. "

```

```

"Your task is to read the task condition "
"and generate a revised, concise, understandable condition with two examples, as well
  as a set of tests.\n"
"Here is an example of the condition and examples:\n"
f"{example}\n"
"Response requirements:\n"
"1. The task conditions should be clear and short.\n"
"2. The examples should be brief, point by point, to help solve the problem.\n"
"3. The list of tests should consist of at least 5, the format is the same as the
  examples."
"Response format (strictly follow)\n"
"{\n\n"
"  \"question\": \"Task condition\",\\n\"
"  \"examples\": [\"Input: ...; Output: ...\"],\\n\"
"  \"tests\": [\"Input: ...; Output: ...\"]\\n\"
"}\n"

```

Далее к промпту добавляются задачи для обработки:

```
f"Processing condition {i}: {questions[i]}\n"
```

3.3. Создание верификатора

Так как методы GRPO и MGPO требуют функцию оценки решения, нужно спроектировать верификатор. Верификатор должен награждать решения, которые запускаются и проходят тесты. Также важно, чтобы верификатор частично награждал за частично выполненные тесты. Ещё важный момент, что код, который может запуститься, лучше чем тот, который не запускается, но это не настолько важный критерий, чтобы награждать так же, как за выполнение тестов. Дополнительно верификатор не должен мешать процессу обучения, ограничивая работу сгенерированных решений.

Ниже описаны подробности реализации верификатора:

- Извлекает код из ответа модели.
- За возможность запуска кода награждать числом $k \in [0, 1]$, за каждый выполненный тест из N имеющихся награждать числом $\frac{1-k}{N}$.

- Запрещает запуск кода с содержанием функциональных библиотек, которые могут вносить изменения в операционную систему. Также блокируется код, содержащий команды запуска другого кода (`exec`, `eval` и др).
- Решение сгенерированной задачи выполняется в отдельном процессе с ограничениями по времени и памяти.

3.4. Обучение модели GPT2

3.4.1. Общие настройки

Поскольку в трёх экспериментах будет обучаться одна модель, были выбраны общие настройки, способные к улучшению результата обучения.

- **Ожидаемый ответ:** модель должна генерировать сначала рассуждение, а затем код. Это достаточно сложная задача для модели с таким количеством весов (примерно 120 миллионов параметров) и таким контекстным окном (1024 токена).
- **Структура промпта:** для хорошего обучения модели нужна чёткая структура промпта, поэтому он выглядит следующим образом: `<|question|> ... <|example|> ... <|reasoning|>`, а модель отвечает в формате: `reasoning + <|code|> + solution`, где `<|question|>`, `<|example|>`, `<|reasoning|>`, `<|code|>` — специальные токены. Пропуск этого шага может существенно снизить показатели метрик модели после обучения.
- **Аккумуляция градиентов:** поскольку GPT2 — модель, построенная на трансформере, нужно использовать большой размер пакета данных при градиентном спуске, иначе градиент будет шумным и модель не сможет прийти к оптимуму. Но обработка даже с помощью GPT2 требует большого количества видеопамяти, и такой пакет данных такого размера не поместится в видеокарту,

с видеопамятью 16 Гб, поэтому будет использоваться аккумуляция градиентов. Также плохим вариантом будет аккумулировать весь датасет, т.к. обучение модели будет очень медленным.

- **Взвешивание токенов:** разные токены при обучении по-разному важны. Рассуждения имеют достаточно схожий паттерн и, если поменять в нём слова местами, то смысл зачастую останется тем же. При генерации кода наоборот важна чёткая последовательность, поэтому токенам кода логичней присвоить больший вес, чем токенам рассуждения. Также важно сгенерировать вовремя специальный токен `<|code|>`, потому что при использовании модели может генерировать бесконечно большие рассуждения, заполняя полностью своё контекстное окно.
- **Оптимизатор и планировщик:** при обучении будут использоваться оптимизатор AdamW[3], он предотвращает переобучение, а также благодаря адаптивной скорости и моменту может выбираться из локальных минимумов. В качестве планировщика использовался планировщик с косинусным затуханием и фазой разогрева: такое сочетание хорошо себя зарекомендовало и используется во многих работах.
- **Метрики:** для оценки результатов обучения будут использоваться 2 метрики: Pass@1 и Pass@3, так как всего было 3 домена и это соответствует статье.

3.4.2. Первый эксперимент: SFT

Данный этап можно рассматривать как стартовую версию. Для обучения будет использован обработанный набор данных APPS, описанный в разделе 3.2.1. Для валидации будет использоваться набор данных, собранный из доменных наборов данных, описанных в разделе 3.2.2. Валидация будет проходить классическим способом при обучении: подсчёт значения функции потерь при генерации токена. Сохранять будем

модель, которая показала лучшее значение функции потерь на валидационных данных.

3.4.3. Второй эксперимент: SFT + GRPO

Первая фаза SFT в этом эксперименте полностью идентична предыдущему, поэтому для фазы RL будут взяты веса из стартовой версии.

Для фазы GRPO будет использоваться обработанный набор данных, описанный в разделе 3.2.3. Итоговая модель будет проверена и сохранена.

3.4.4. Третий эксперимент: SSP

В этом эксперименте обе фазы отличаются от предыдущих.

В фазе SFT валидация будет происходить на задачах из оценочных наборов. Для каждого оценочного набора данных будет свой подсчёт метрики: для этого семплируется N задач, и на них считается метрика Pass@3, веса модели сохраняются, если результат оказался лучшим.

После этого происходит слияние экспертных моделей в одну, их веса усредняются.

Затем идёт фаза MGPO. От GRPO этот этап отличается способом вычисления функции потерь, но сам процесс обучения такой же, как в предыдущем: учимся на том же наборе данных и после сохраняем модель.

4. Эксперимент

В данном разделе будут описаны конкретные настройки экспериментов и их результаты.

4.1. Условия эксперимента

Эксперименты по обучению и генерации наборов данных проходили на платформе Kaggle. Ниже приведены технические характеристики вычислительной среды:

- CPU: Intel Xeon 2.00GHz
- CPU count: 4
- GPU: Tesla P100-PCIE-16GB
- GPU count: 1
- RAM: 31.35 GB

4.2. Исследовательские вопросы

В ходе экспериментов нужно было ответить на следующие исследовательские вопросы.

RQ1 : Будет ли прирост метрики Pass@1, используя метод SSP по сравнению с обычным SFT?

RQ2 : Будет ли прирост метрики Pass@1, используя метод SSP по сравнению со связкой SFT + GRPO?

4.3. Метрики

В качестве метрик в работе используются Pass@1 и Pass@3, поскольку именно эти метрики используются в статье в качестве основных показателей качества.

4.4. Результаты

Ниже представлены результаты обучения моделей.

Таблица 1: Значения метрик по методам обучения

Метрика	SFT	SFT + GRPO	SFT + MGPO
Pass@1	$6.9 \cdot 10^{-4}$	$6.9 \cdot 10^{-4}$	0
Pass@3	$2.1 \cdot 10^{-3}$	$2.1 \cdot 10^{-3}$	0

4.4.1. Результаты обучения методом SFT

На рисунке 3 представлен график обучения модели методом тонкой настройки.

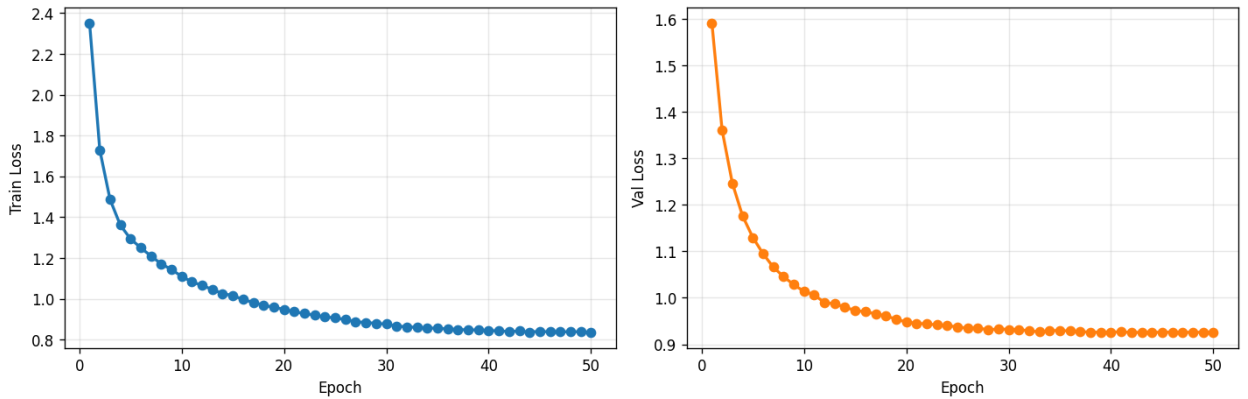


Рис. 3: Графики функций потерь на тренировочном и тестовом наборах данных от количества пройденных эпох.

4.4.2. Результаты обучения методом SFT + GRPO

В качестве фазы SFT использовались веса модели из первого эксперимента.

Фаза GRPO оказалась неудачной: модель не могла набрать больше 0 по метрике Pass@1, поэтому веса модели не обновлялись.

4.4.3. Результаты обучения методом SSP

На рисунке 4 представлен график обучения фазы SFT с отмеченными экспертными моделями. После этого их веса были усреднены.

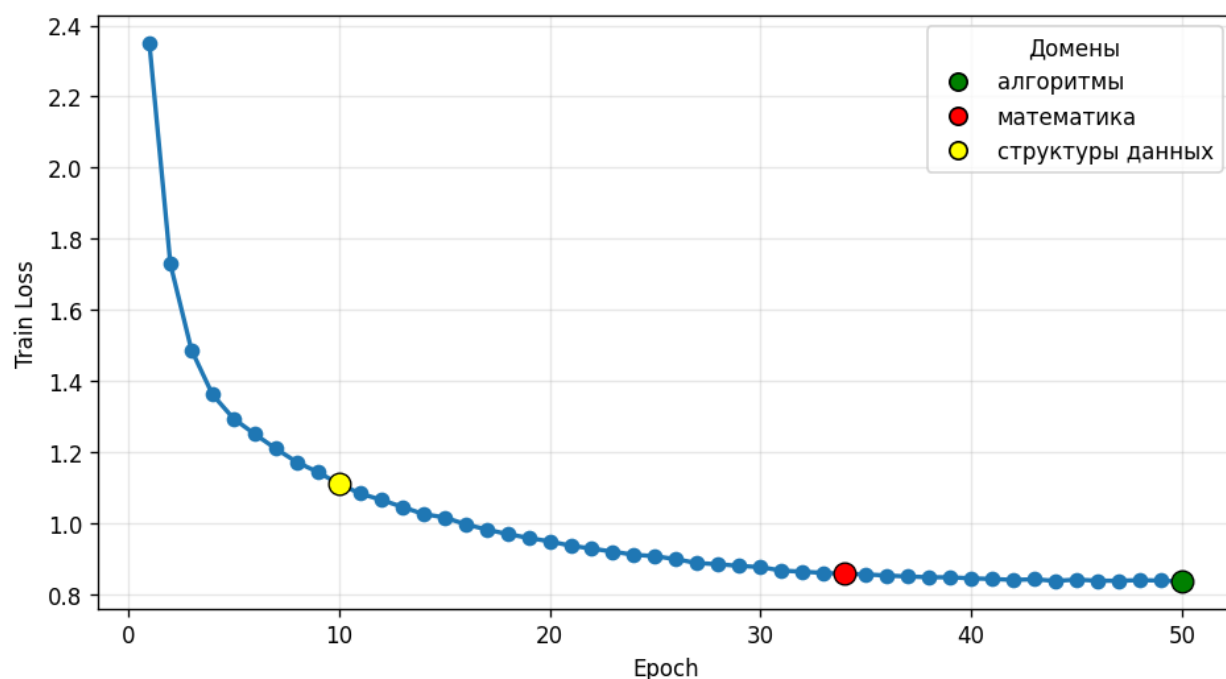


Рис. 4: Графики функций потерь на тренировочном наборе данных от количества пройденных эпох с экспертными моделями.

Фаза MGPO так же GRPO была провалена: модель также считала метрику 0, и значение не обновлялось.

4.4.4. RQ1

Из таблицы 1 видно, что предложенный метод в статье показал хуже результат, чем тонкая настройка.

4.4.5. RQ2

Из таблицы 1 видно, что предложенный метод в статье показал хуже результат, чем комбинация SFT + GRPO.

4.5. Анализ результатов

Методы GRPO и MGPO не дали никакого прироста, потому что модель на наборе данных для RL показывала значения по метрикам 0. Это значит, что модель после фазы SFT была неподготовленной к применению RL: она показывала слишком низкие результаты по метрикам, что вид-

но из таблицы 1. Также при ручной проверке решений модель зачастую всё же ошибалась в решении задачи.

Есть несколько предположений, почему так произошло:

- Модель для эксперимента была выбрана с наименьшим числом параметров из имеющихся. Задача генерации рассуждения и решения на Python изначально была сложной для этой модели.
- Контекстное окно GPT2 сильно ограничивает длину рассуждений. В больших языковых моделях рассуждение — длинная цепочка мыслей, описывающая задачу и процесс решения. В данной работе подготавливались наборы данных с небольшим рассуждением, которое лишь наталкивало на само решение.
- Маленький объём наборов данных. Для фазы SFT использовался набор данных, состоящий из 3 тысяч задач. При увеличении набора данных и при более длительном обучении, модель могла бы выявить больше закономерностей и улучшить результат. Также для фазы RL набор данных является небольшим, что может в дальнейшем ухудшить потенциальный результат.
- Выборка из 15 задач для оценки метрики. Такое решение было принято для ускорения обучения, но такой подход в плохо обученных моделях иногда позволяет сохранять чекпоинты, для которых выборка задач оказалась удачной. Поэтому фаза со слиянием моделей в одну только ухудшила результат. К тому же из текста статьи можно узнать, что фаза MGPO должна была устранить ухудшение при слиянии и ещё улучшить результат.

Скорее всего, исправив эти недостатки, можно полнее раскрыть потенциал метода SSP SSP и улучшить результаты.

Заключение

В данной работе был проведён анализ статьи о методе обучения SSP, созданы наборы данных для проверки эффективности предложенного метода в задаче генерации решений на языке Python, а также проведены сами эксперименты. В процессе работы были выполнены поставленные задачи.

1. Был проведён обзор предложенной статьи, метода обучения модели VibeThinker-1.5B, а также дополнительного алгоритма в фазе RL, который участвовал в сравнении. (Подробнее 2.2)
2. Были подготовлены наборы данных для задачи генерации решений на языке Python. В работе использовались наборы для тонкой настройки, оценочные наборы и набор для фазы RL. (Подробнее 3.2)
3. Модель GPT2 была дообучена методом SFT. (Подробнее 3.4.2 и 4.4.1)
4. Модель GPT2 была дообучена комбинацией алгоритмов SFT и GRPO. (Подробнее 3.4.3 и 4.4.2)
5. Модель GPT2 была дообучена предложенным методом SSP. (Подробнее 3.4.4 и 4.4.3)
6. По результатам обучения видно, что метод SSP не показал ожидаемой эффективности, т.к. модель после этапа тонкой настройки показывали низкий результат по метрикам Pass@1 и Pass@3. В итоге стандартный метод и метод с использованием GRPO показали более высокие значения метрик, чем SSP. (Подробнее 4.4)

Исходный код данной работы: https://github.com/VadosikRRR/SSP_TINY_MODELS

Список литературы

- [1] DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning / Daya Guo, Dejian Yang, Haowei Zhang et al. — 2025. — URL: <https://www.nature.com/articles/s41586-025-09422-z> (дата обращения: 10 марта 2026 г.).
- [2] DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models / Zhihong Shao, Peiyi Wang, Qihao Zhu et al. — 2024. — URL: <https://arxiv.org/abs/2402.03300> (дата обращения: 10 марта 2026 г.).
- [3] Ilya Loshchilov Frank Hutter. Decoupled Weight Decay Regularization. — 2017. — URL: <https://arxiv.org/abs/1707.06347> (дата обращения: 17 марта 2026 г.).
- [4] Language Models are Unsupervised Multitask Learners / Alec Radford, Jeffrey Wu, Rewon Child et al. — 2019. — URL: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf (дата обращения: 10 марта 2026 г.).
- [5] Mistral 7B / Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch et al. — 2023. — URL: <https://arxiv.org/abs/1707.06347> (дата обращения: 17 марта 2026 г.).
- [6] Proximal Policy Optimization Algorithms / John Schulman, Filip Wolski, Prafulla Dhariwal et al. — 2017. — URL: <https://arxiv.org/abs/1707.06347> (дата обращения: 17 марта 2026 г.).
- [7] Qwen2.5-Math Technical Report: Toward Mathematical Expert Model via Self-Improvement / An Yang, Beichen Zhang, Binyuan Hui et al. — 2024. — URL: <https://arxiv.org/abs/2409.12122> (дата обращения: 10 марта 2026 г.).
- [8] Tiny Model, Big Logic: Diversity-Driven Optimization Elicits Large-Model Reasoning Ability in VibeThinker-1.5B / Sen Xu, Yi Zhou,

Wei Wang et al. — 2025. — URL: <https://arxiv.org/abs/2511.06221>
(дата обращения: 10 марта 2026 г.).